HTML5 - Integración Multimedia

Índice

1. Imágenes e iconos:
    a. Imágenes "con enlaces": <map> y <area>
    b. Nuevo control <picture>
    c. Gráficos SVG <svg>
    d. Iconos con librería Font Awesome
2. Audio y Video
    a. Control <audio>
    b. Control <video>
3. Otros objetos incrustados
    a. Marcos <iframe>
    b. Lienzo de diseño con javascript <canvas>
    c. Otros objetos

1. Inserción de imágenes

a. Imágenes interactivas con enlaces: <map> y <area>

En HTML, las imágenes se insertan con la etiqueta <img>, donde podemos especificar la ruta del archivo, el ancho y un texto alternativo que se muestra si la imagen no se carga.

```
<img src="./img/perro.gif" width="400" alt="imagen de un perro" >
```

Nota: es bueno poner el ancho predefinido (width) en HTML, por si hay problema con la carga del CSS.

Para asociarle un mapa de enlaces clicables dentro de la propia imagen usamos atributo **usemap**

```
<img src="mar.gif" width="400" usemap="#nommapa" alt="paisaje marino " >
```

Posteriormente se define el mapa en el mismo documento HTML con <map> y <area>.
Ejemplo:

```
<img src="imagen.gif" width="400px" usemap="#mimapa" alt="foto retrato">
<map name="mimapa">
  <area shape="rect" coords="0,0,50,50" href="pagina1.html" target="_blank" >
  <area shape="circle" coords="75,75,25" href="http://www.terra.es" >
  <area shape="poly" coords="0,20,30,25,40,10" href="pag2.htm#secc1" >
</map>
```

La definición de áreas se hace con <area>, indicando shape (rect, circle, poly) y sus coordenadas.

Nota: el <map> se puede poner después del element img, o bien al final del fichero html. Lo que vincula <img> con su <map> es el nombre (name) que le demos, que debe ser el referenciado por usemap.

Más info: *https://www.w3schools.com/tags/tag_map.asp*

Nota: aunque <map> es una etiqueta permitida en HTML5, **no se recomienda su uso** ya que no funciona bien en páginas web responsivas, donde la imagen cambia de tamaño (ya que los shape se expresan en coordenadas fijas y absolutas). Por tanto, **en este curso no las usaremos**.

b. Control <picture> y <source>

Picture es como la versión plus de <img>

<picture> permite especificar distintas imágenes según el dispositivo, útil para diseño responsive.

Ejemplo:

```
<picture>
  <source media="(min-width: 1020px)" srcset="./img/logoenorme.png">
  <source media="(min-width: 800px)" srcset="./img/logogrande.png">
  <img src="./img/normal.png" alt="logo.png">
</picture>
```

Explicación: *Navegador mira si viewport tiene 1024px o más. En ese caso carga la imagen "logoenorme.png". Si no tiene esa medida, intenta cargar el segundo source: mira si viewport tiene 800px o más. Si se cumple carga "logogrande.png". Si falla éste source, entonces muestra el elemento img.* ***En picture es una práctica habitual siempre poner al final un elemento img*** *(por si navegador no acepta picture).*

- *Para que funcione bien el elemento picture y sus "mediaqueries" hay que usar este meta:* `<meta name="viewport" content="width=device-width, initial-scale=1">`
- *El elemento picture no tiene ningún atributo específico.*
- *Atributos de source:*
  - *srcset (obligatorio)*
    - *Ruta de la imagen a la que se hace referencia . Puede indicar una lista de rutas separadas por comas y que incluyan el sufijo que indica la densidad de píxeles Ejemplo: srcset="pollo.png, polloHD.png 2x".*
  - *media (opcional)*
    - *Permite indicar cualquier media query que sea válido en el selector @media de CSS3 Ejemplo media="(max-width: 30em")* → *@media de CSS3 lo veremos en otro tema*
  - *sizes (opcional)*
  - *Acepta cualquier valor que describa la anchura de la imagen (ejemplo sizes="100vw")*
  - *type (opcional)*
  - *Acepta como valor cualquier tipo MIME estándar (type="image/webp")*

Más info: https://www.w3schools.com/tags/tag_picture.asp

c. Inserción de gráficos SVG <svg>

SVG (Scalable Vector Graphics) es un lenguaje basado en XML que permite definir gráficos vectoriales directamente dentro de una página HTML. A diferencia de JPG o PNG, los gráficos SVG pueden ampliarse o reducirse sin pérdida de calidad.

La etiqueta <svg> actúa como contenedor de los elementos gráficos. Dentro de ella pueden definirse rectángulos (<rect>), círculos (<circle>), líneas (<line>), polígonos (<polygon>) y texto (<text>).

```
<svg width="200" height="120">
  <rect x="10" y="10" width="180" height="100" fill="lightblue"
                                        stroke="navy" stroke-width="3" />
</svg>
```

Ejemplo de círculo:

```
<svg width="120" height="120">
  <circle cx="60" cy="60" r="50" fill="yellow" stroke="black" stroke-
width="2" />
</svg>
```

Tutorial completo sobre SVG: **https://www.w3schools.com/graphics/svg_intro.asp**
Nota: no estudiar etiquetas SVG, simplemente conviene saber qué es SVG y saber buscar recursos en repositorios (o saber convertir de logo png/jpg a fichero .svg , cosa que veremos en el futuro).

De momento aquí os dejo unos cuantos repositorios donde buscar iconos y otros assets:
https://www.svgrepo.com https://allsvgicons.com https://freeicons.io https://www.flaticon.com

Los ficheros .svg, también pueden mostrarse dentro de elemento <img> o <picture>:

```
  <img src="logosol.svg" width="400px" alt="logo de un sol" >
```

Ventajas de SVG:

- Se adapta a cualquier resolución sin perder calidad.
- Puede modificarse mediante CSS.
- Permite animaciones e interacción con JavaScript.
- Ocupa poco espacio en muchos casos.
- Cada elemento forma parte del DOM y puede recibir eventos.

Enlaces dentro de un SVG

Cada figura SVG puede actuar como un enlace mediante la etiqueta <a>.

```
<svg width="250" height="100">
  <a href="https://www.google.es">
    <rect x="20" y="20" width="200" height="50" fill="lightgreen" stroke="black"/>
    <text x="70" y="52">Ir a Google</text>
  </a>
</svg>
```

Al hacer clic sobre el rectángulo o el texto se abrirá la URL indicada.

Manipulación de SVG mediante CSS *(no estudiar, ya lo veremos en tema CSS)*

Una de las principales ventajas de SVG es que sus elementos pueden modificarse mediante CSS igual que cualquier elemento HTML.

```
<svg width="200" height="120">
  <circle class="pelota" cx="60" cy="60" r="40" />
</svg>
```

.pelota {
fill: orange;
stroke: black;
stroke-width: 2;
}
	.pelota:hover {
	fill:red;
	}

Al pasar el ratón sobre el círculo, cambia de color gracias a la pseudoclase :hover. No estudiar: son un par de ejemplos para que veáis que es posible formatear CSS	

El atributo viewBox

El atributo viewBox define el sistema de coordenadas interno del gráfico SVG. Gracias a él, el dibujo puede escalarse correctamente independientemente del tamaño visual que tenga en pantalla. SVG está diseñado para renderizarse a cualquier tamaño sin pérdida de calidad y utiliza un sistema de coordenadas propio.

Sintaxis: `<svg width="400" height="200" viewBox="0 0 200 100">`

viewBox=**"x y ancho alto".** En este caso: Área de dibujo interna mide 200 × 100 unidades, gráfico se mostrará ocupando 400 × 200 píxeles. El navegador reescalará automáticamente elementos

Si queremos un svg "responsive": `<svg viewBox="0 0 200 100" width="100%">`

El SVG ocupará todo el ancho disponible manteniendo sus proporciones.

Nota: En proyectos modernos se recomienda utilizar viewBox, ya que facilita el diseño adaptable para móviles y tabletas.

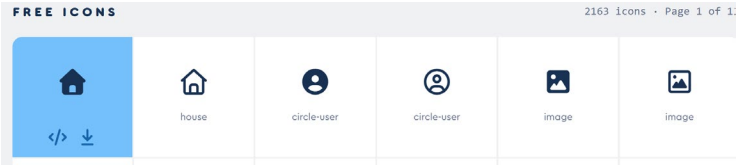
d. Librería Font Awesome: Insertar iconos de forma profesional

Font Awesome es una biblioteca de iconos que permite incorporar miles de iconos mediante clases CSS sin necesidad de descargar imágenes individuales. La librería se puede bajar de la página oficial, o bien se puede referenciar a alguna que se publique en un CDN (Content Delivery Network, como https://cdnjs.com/ o https://www.jsdelivr.com/). Para ello, entramos en el CDN y copiamos etiqueta link y la pegamos en cabecera del html donde queramos usar iconos FontAw:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/7.3.1/css/all.min.css">
```

Nota: siempre que podamos usamos versión .min. (que significa que ocupa menos espacio).

Luego, para poner iconos en nuestro documento, usamos la etiqueta <i> y la clase que indica el icono a usar. FontAwesome tiene millones de iconos, pero muchos de pago. Los gratis los podéis encontrar aquí: https://fontawesome.com/search?ic=free-collection



En cada icono, nos deja **bajarlo en SVG,** o bien **copiar su etiqueta <i> con la clase adecuada**:

Ejemplos:

```
<i class="fa-solid fa-house"></i>
<i class="fa-solid fa-user"></i>
```

Los iconos se pueden aparecer dentro de botones:

```
<button><i class="fa-solid fa-cart-shopping"></i> Comprar</button>
```

Los iconos se pueden **personalizar** mediante CSS o bien mediante clases propias de FontAwesome:

```
<i class="fa-solid fa-house fa-xl fa-rotate-90"></i>
```

```
i{
    color:red;
}
```

```
i:hover{
   color: blue;
}
```

En este apartado explica bastantes cosas sobre la personalización de iconos:

https://docs.fontawesome.com/web/style/

Ventajas de Font Awesome:

- Gran colección de iconos.
- Escalables sin pérdida de calidad.
- Fácil personalización mediante CSS.
- Mejora la usabilidad y la interfaz.
- Compatible con los principales navegadores.

2. Inserción de Audio y Video

a. Inserción de audio <audio> y <source>

La etiqueta <audio> permite reproducir sonidos directamente en el navegador sin necesidad de plugins externos. Se pueden definir varias fuentes con <source> para asegurar compatibilidad.

Ejemplo:

```
<audio autoplay controls>
  <source src="./musica/bailando.mp3" type="audio/mpeg" >
  <source src="./musica/bailando.ogg" type="audio/ogg" >
  Navegador no soporta este elemento de audio
</audio>
```

Explicación: *Navegador intentará cargar el primer **source**. Si no puede cargarlo, intenta el 2º. Si falla éste, entonces muestra ese texto. En vez de texto se podría poner una imagen, un enlace, etc.*

- *Sus principales atributos son:*
  - *autoplay: indica que el contenido se ejecutara automáticamente tras su descarga*
  - *loop: indica que el contenido se volverá a ejecutar tras terminar*
  - *controls: : indica si queremos mostrar un reproductor de audio para controlar sonido.*
  - *src="": Si tenemos una fuente única, podemos usar este atributo en vez de <source>*

- *Recomendaciones:*
  - *Para hacer web compatible con todos los navegadores, **siempre poner dos fuentes de audio**: una **ogg** y otra **mp3.** **Aquí podéis hacer conversiones:** **http://www.online-convert.com/es***
  - ***src** siempre apunta a ruta relativa donde se encuentre el archivo de audio*
  - *No poner **autoplay** en audio de página de inicio de un website*
    ***Nota: autoplay** ya está deshabilitado en casi todos los navegadores!*
    - *Es aconsejable poner los controles visibles, atributo **controls** (o controlar con javascript)*

b. Inserción de vídeo <video> y <source>

La etiqueta <video> permite reproducir vídeo directamente en el navegador. Igual que en audio, podemos definir varias fuentes.

Ejemplo:

```
<video controls>
  <source src="discurso.ogv" type="video/ogg" >
  <source src="discurso.mp4" type="video/mp4" >
  Tu navegador no puede visualizar este vídeo.<br >
  <a href="discurso.ogv"> Descarga el vídeo aquí</a></p>
</video>
```

Explicación: *Navegador intentará cargar el primer **source**. Si no puede cargarlo, intenta el 2º. Si falla éste, entonces muestra ese enlace de descarga.*

- *Sus principales atributos son:*
  - ***autoplay**: contenido se ejecutara automáticamente (solo va si ponemos "muted")*
  - ***muted**: salida de audio "muteada"*
  - ***preload**: auto|none si el fichero debe cargarse inicialmente, o al darle al play*
  - *loop: indica que el contenido se volverá a ejecutar tras terminar*
  - *controls: indica si queremos controles (pausa, play…) en el reproductor de vídeo.*
  - *src="": Si tenemos una fuente única, podemos usar este atributo en vez de <source>*
  - *height y width: Tamaño en la pantalla*
  - *poster: imagen que se mostrara cuando el video no este disponible o no sea accesible.*

- *Recomendaciones:*
  - *Para hacer web compatible con todos los navegadores, **siempre poner dos fuentes de video**: una **ogv** y otra **mp4** **http://www.online-convert.com/es***
  - *En atributo **type** de <source>, mejor si ponemos también el **códec** del vídeo*
    http://www.w3.org/wiki/HTML/Elements/video#Examples

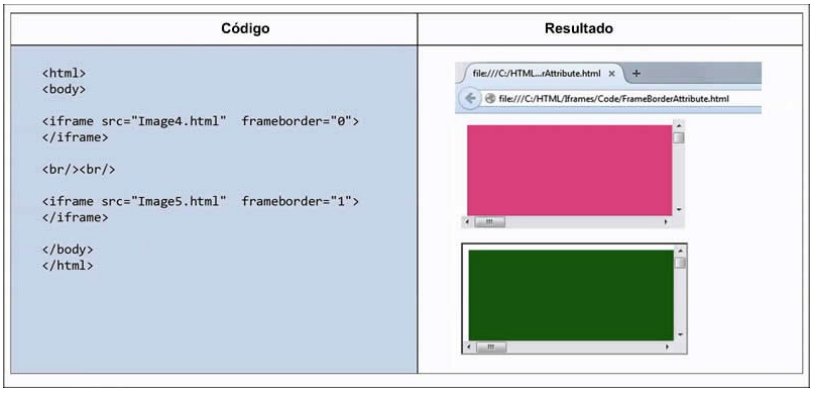
3. Otros elementos incrustados

a. Marcos <iframe>

<iframe> permite definir un área dentro de una página HTML donde se carga otro documento HTML, incluso externo.

Ejemplo básico:

```
<iframe src="http://www.terra.es" width="100%" height="800" sandbox></iframe>
```



Podemos usar enlaces <a> con el atributo target para abrir una página dentro de un iframe con un nombre específico.

Primero definimos el iframe con un atributo name:

```
<iframe src="doc1.html" name="marco1" width="600" height="400"></iframe>
```

Después, creamos enlaces que abran en ese marco:

```
<a href="doc2.html" target="marco1">Abrir doc2 en el marco</a>
<a href="doc3.html" target="marco1">Abrir doc3 en el marco</a>
```

De este modo podemos tener un área de la página donde se carga contenido dinámicamente según los enlaces que se pulsen.

Nota: recordad que otros usos de target, eran target="_blank" o target="_self" *(por defecto)*

El atributo sandbox

El atributo sandbox restringe el contenido: no ejecuta scripts ni redirecciones salvo que se permitan expresamente con valores como allow-forms, allow-scripts, allow-top-navigation, allow-same-origin.

Ejemplo con sandbox más permisivo que poner solo `sandbox`*(sin valores)*:

```
<iframe src="externo.html" name="marco2" sandbox="allow-scripts allow-same-origin"></iframe>
```
=> En ese caso permitimos que ejecute javascript

b. Objeto <canvas> *(no estudiar)*

El elemento HTML <canvas> es un lienzo en blanco que se utiliza para dibujar gráficos, animaciones y juegos en una página web mediante secuencias de comandos, por lo general con JavaScript.

¿Cómo funciona Canvas?

El elemento <canvas> por sí solo es solo un contenedor o área rectangular en la página. Para pintar formas, líneas, texto o imágenes dentro de él, debes usar la API de Canvas a través de JavaScript.

Ejemplo básico de uso

Para crear un rectángulo rojo sencillo, combinas HTML y JavaScript de la siguiente manera:

```
<canvas id="miLienzo" width="200" height="100"></canvas>

<script>
      const canvas = document.getElementById("miLienzo");
      const ctx = canvas.getContext("2d");
      ctx.fillStyle = "red";
      ctx.fillRect(10, 10, 100, 50);
</script>
```

Nota: Canva se programa todo con JavaScript, por lo que no vamos a ahondar más en el tema, ya que es más propio de otros módulos donde se estudia programación de lado cliente. No obstante aquí tenéis un tutorial: https://developer.mozilla.org/es/docs/Web/API/Canvas_API/Tutorial

Diferencia de <svg> con <canvas>:
SVG mantiene cada objeto como un elemento independiente dentro del documento, mientras que <canvas> es un único elemento, y dentro de él se dibuja píxel a píxel mediante JavaScript.

c. Otros objetos incrustados *(no estudiar)*

HTML5 recomienda usar <audio>, <video>, <iframe> y <canvas> para control multimedia. Mantiene <embed> y <object> para compatibilidad con contenidos antiguos como Flash o widgets.

Ejemplo de Flash (tecnología de clips multimedia popular en los 2000, ya en desuso):

```
<object width="400" height="400" data="./media/serverside.swf"></object>
```

Ejemplo de vídeo de YouTube antiguo con <embed> : (Ahora Youtube videos usan iframe)

```
<embed width="420" height="345" src="http://www.youtube.com/v/nAKYVMCkB4I"
type="application/x-shockwave-flash" >
```

Nota: aunque son etiquetas permitidas en HTML5, **no se recomienda su uso,** hacen refencia a tecnologías en desuso, y se han mantenido en el estándar solo por compatibilidad.

NOTA: *Veremos más cosas de multimedia cuando veamos CSS3: transform, transition, animation…*